

UFRN - Universidade Federal do Rio Grande do Norte

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 16 - PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

1. Enunciado

Implemente um programa MPI que calcule o produto $y = A \cdot x$, onde A é uma matriz $M \times N$ e x é um vetor de tamanho N . Divida a matriz A por linhas entre os processos com `MPI_Scatter`, e distribua o vetor x inteiro com `MPI_Bcast`. Cada processo deve calcular os elementos de y correspondentes às suas linhas e enviá-los de volta ao processo 0 com `MPI_Gather`. Compare os tempos com diferentes tamanhos de matriz e número de processos.

2. Fundamentação Teórica

2.1 Multiplicação matriz-vetor e decomposição por linhas

O produto $y = A \cdot x$, com A de dimensão $M \times N$ e x de dimensão N , resulta em um vetor y de dimensão M , onde cada elemento $y[i]$ é o produto interno entre a linha i de A e o vetor x . Como o cálculo de cada $y[i]$ depende apenas da linha correspondente de A e do vetor x completo, o problema é embarçosamente paralelo por linhas: distribuindo blocos de linhas de A entre P processos, cada processo pode calcular seu subconjunto de y de forma totalmente independente, sem comunicação entre processos durante o cálculo — apenas na distribuição inicial dos dados e na coleta final dos resultados.

2.2 Operações coletivas de MPI utilizadas

MPI_Scatter: distribui blocos contíguos de um buffer presente no processo raiz para todos os processos do comunicador, um bloco de tamanho igual por processo. Como A é armazenada em row-major, um bloco de `local_M = M/P` linhas corresponde a um segmento contíguo de `local_M × N` doubles — por isso M precisa ser divisível por P nesta implementação.

MPI_Bcast: replica o vetor x (tamanho N) do processo raiz para todos os processos, já que todo processo precisa do vetor completo para calcular o produto interno de suas linhas locais.

MPI_Gather: operação inversa ao Scatter — cada processo envia seu subvetor local_y (local_M elementos) e o processo raiz os concatena, na ordem dos ranks, reconstruindo o vetor y completo.

As três operações são implementadas internamente com algoritmos em árvore (tipicamente $O(\log P)$ passos de comunicação para o padrão de dados envolvido), o que as torna mais eficientes que um laço de Send/Recv ponto a ponto equivalente, mas ainda assim implicam um custo fixo de latência e sincronização que não desaparece quando o volume de dados por processo diminui.

2.3 Métricas de desempenho: Speedup e Eficiência

Para avaliar o ganho de paralelizar o problema mantendo o tamanho fixo (strong scaling), usam-se duas métricas clássicas:

Speedup: $S(P) = T(1) / T(P)$, onde $T(1)$ é o tempo de execução com 1 processo e $T(P)$ o tempo com P processos. O speedup ideal é linear, $S(P) = P$.

Eficiência: $E(P) = S(P) / P$, que mede a fração do ganho ideal efetivamente obtida. Segundo a Lei de Amdahl, a eficiência tende a cair à medida que P cresce sempre que existe uma parcela não-paralelizável ou um custo fixo de comunicação/sincronização que não escala com P — exatamente o comportamento esperado aqui, já que o overhead de Scatter/Bcast/Gather não desaparece quando o trabalho local por processo diminui.

3. Implementação

O programa multMxV.c (Anexo A) segue exatamente a estratégia pedida no enunciado:

- O processo 0 gera a matriz A ($M \times N$) e o vetor x (N) com uma semente fixa, garantindo reprodutibilidade entre execuções;
- MPI_Scatter distribui local_M = M/P linhas de A para cada processo (o programa verifica $M \% P == 0$ e aborta com mensagem de erro caso contrário, já que MPI_Scatter exige blocos de tamanho igual);
- MPI_Bcast distribui o vetor x completo a todos os processos;
- cada processo calcula $\text{local_y}[i] = \sum_j \text{local_A}[i][j] \cdot x[j]$ para suas local_M linhas;
- MPI_Gather recolhe os local_y de cada processo no vetor y completo do processo 0;
- o tempo é medido com MPI_Wtime entre duas MPI_Barrier, envolvendo Scatter + Bcast + cálculo local + Gather, e é repetido reps vezes (30 no experimento) para obter uma média estável;
- uma flag --verify recalcula y de forma serial no processo 0 e imprime o erro absoluto máximo em relação ao resultado paralelo, permitindo validar a corretude do algoritmo distribuído sem contaminar as medições de tempo (o verify roda fora do laço de repetições cronometrado).

A saída do programa é uma linha no formato CSV — nprocs,M,N,reps,tempo_medio_s — o que permite agregar diretamente os resultados de múltiplas execuções em um único arquivo para análise.

4. Experimento

4.1 Ambiente de execução

As medições foram realizadas no cluster NPAD, partição amd-512, através do job SLURM script.sh (Anexo B), que:

- reserva 1 nó inteiro, com exclusividade (--exclusive), e até 32 tarefas (--nodes=1 --ntasks=32);
- compila o código com mpicc -O3 -o multMxV multMxV.c -lm;
- varre TAMANHOS = (512, 1024, 2048, 4096, 8192) para M=N, e NPROCS_LIST = (1, 2, 4, 8, 16, 32);
- executa cada uma das 30 combinações (5 tamanhos × 6 contagens de processos — todas válidas, já que 512, 1024, 2048, 4096 e 8192 são divisíveis por 1, 2, 4, 8, 16 e 32) com reps=30, concatenando as linhas de saída em resultados_tarefa16.csv.

5. Resultados

A tabela abaixo reúne o tempo médio (30 repetições), o speedup $S(P) = T(1)/T(P)$ e a eficiência $E(P) = S(P)/P$ para as 30 combinações de tamanho de matriz (M=N) e número de processos (P) executadas.

M = N	P	Tempo médio (ms)	Speedup S(P)	Eficiência E(P)
512	1	0.3480	1.000	100.00%
512	2	0.3019	1.153	57.64%
512	4	0.1961	1.775	44.37%
512	8	0.1449	2.402	30.02%
512	16	0.1402	2.482	15.51%
512	32	0.1837	1.895	5.92%
1024	1	1.3569	1.000	100.00%
1024	2	0.9321	1.456	72.79%
1024	4	0.7911	1.715	42.88%
1024	8	0.8028	1.690	21.13%
1024	16	0.3432	3.954	24.71%
1024	32	0.2982	4.550	14.22%
2048	1	6.0188	1.000	100.00%
2048	2	4.0664	1.480	74.01%
2048	4	2.7083	2.222	55.56%
2048	8	2.6548	2.267	28.34%
2048	16	0.9067	6.638	41.49%
2048	32	0.8184	7.354	22.98%
4096	1	28.2608	1.000	100.00%
4096	2	18.9927	1.488	74.40%
4096	4	12.4815	2.264	56.61%
4096	8	10.6600	2.651	33.14%
4096	16	2.9439	9.600	60.00%
4096	32	2.1904	12.902	40.32%
8192	1	113.7086	1.000	100.00%
8192	2	78.2637	1.453	72.64%
8192	4	59.8528	1.900	47.50%
8192	8	54.6591	2.080	26.00%
8192	16	22.3002	5.099	31.87%
8192	32	18.2141	6.243	19.51%

As quatro figuras a seguir apresentam a mesma informação de forma visual, para facilitar a comparação entre os dois eixos pedidos no enunciado — tamanho de matriz e número de processos.

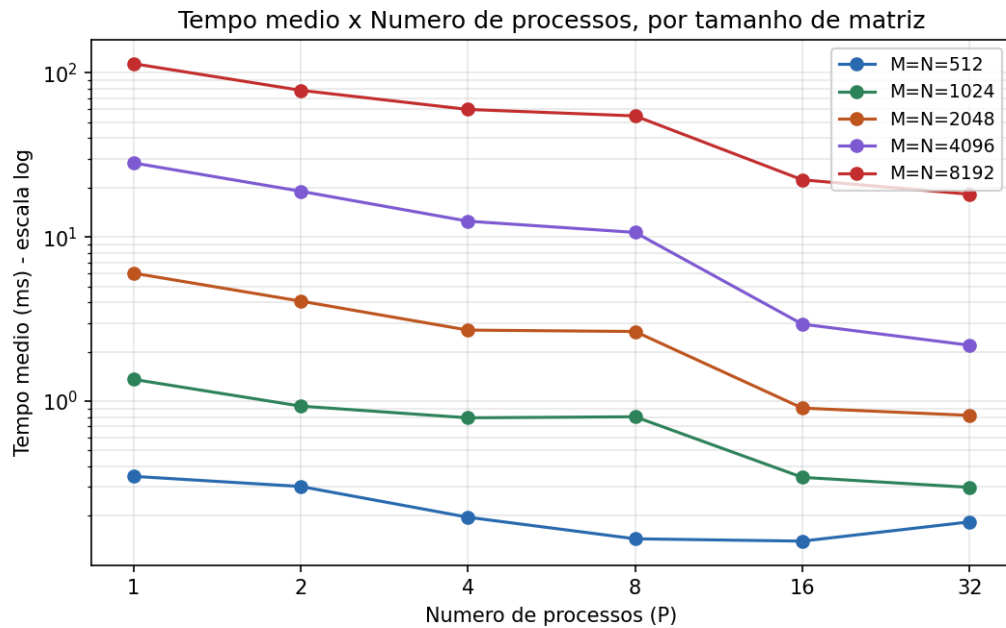


Figura 1 — Tempo médio (escala log) x número de processos, uma curva por tamanho de matriz.

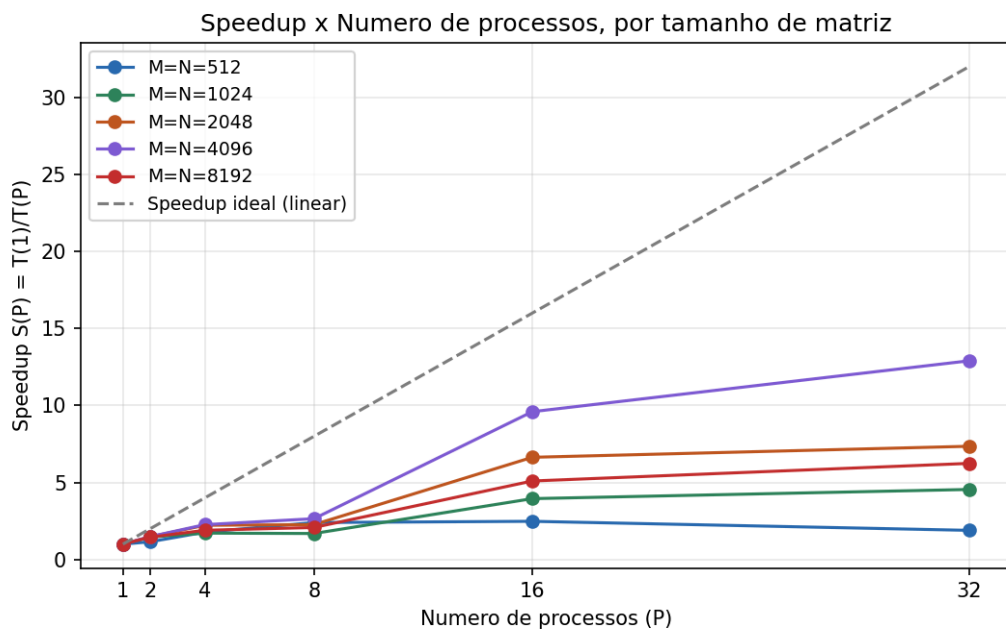


Figura 2 — Speedup x número de processos, uma curva por tamanho de matriz, com a reta de speedup ideal.

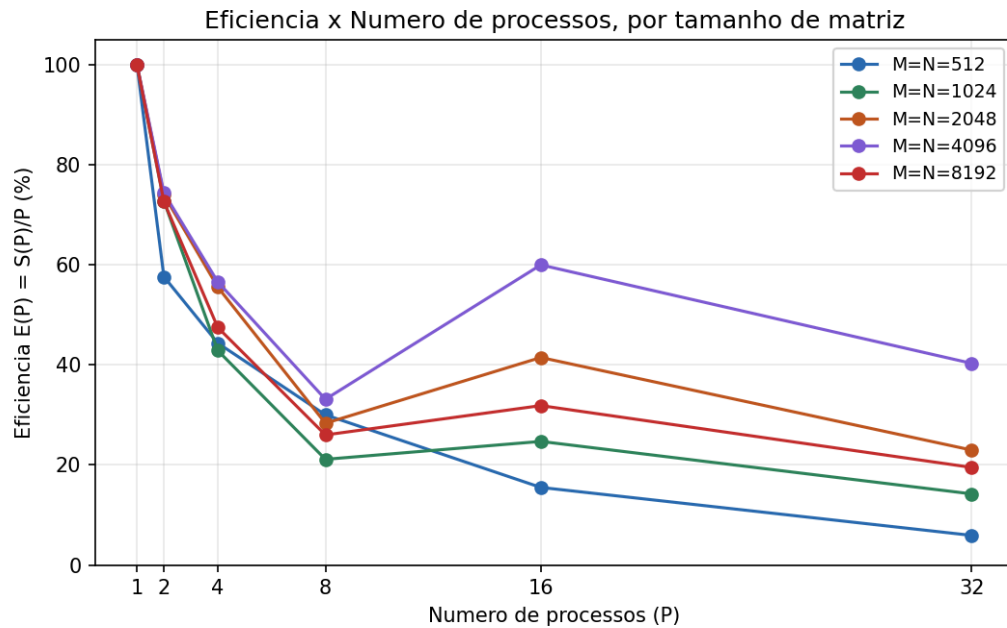


Figura 3 — Eficiência x número de processos, uma curva por tamanho de matriz.

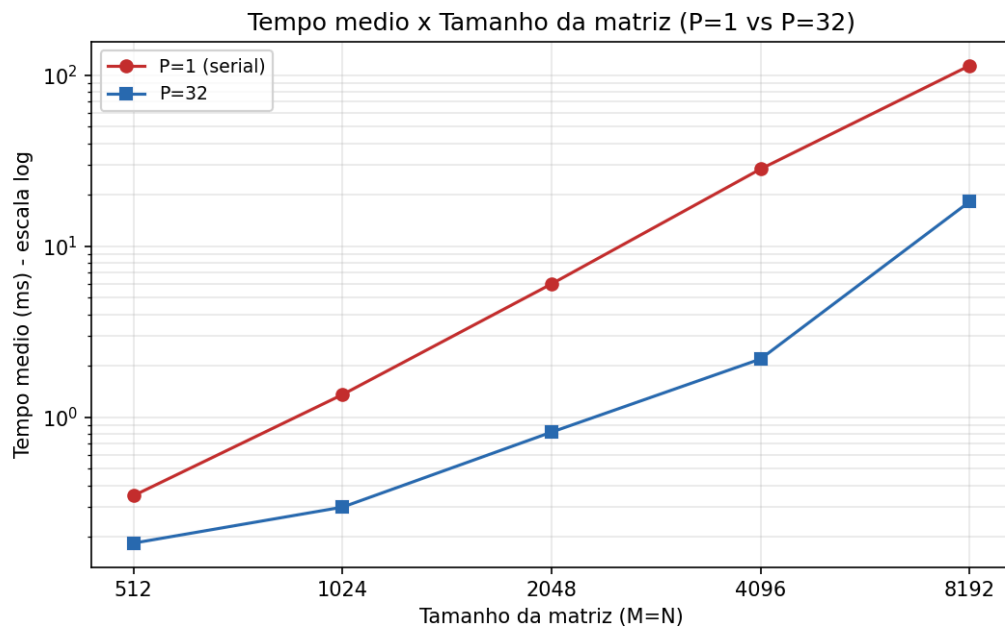


Figura 4 — Tempo médio x tamanho da matriz, comparando execução serial (P=1) e com P=32 (escala log-log).

6. Discussão

6.1 Efeito do tamanho da matriz (P fixo)

Com $P=1$, o tempo cresce de 0,35 ms ($N=512$) para 113,71 ms ($N=8192$) um fator de ≈ 327 para um aumento de 16 vezes em N . Isso é exatamente o esperado o trabalho de $y=A \cdot x$ é $O(M \cdot N) = O(N^2)$ quando $M=N$, então dobrar N deveria multiplicar o tempo por ≈ 4 . Conferindo razão a razão: $512 \rightarrow 1024$ (3,90 $\times$), $1024 \rightarrow 2048$ (4,44 $\times$), $2048 \rightarrow 4096$ (4,70 $\times$) e $4096 \rightarrow 8192$ (4,02 $\times$) todas próximas

de 4x, confirmando que a implementação segue a complexidade quadrática esperada para a multiplicação matriz-vetor densa (Figura 4).

6.2 Efeito do número de processos (M=N fixo)

Para todo tamanho de matriz, o tempo cai de P=1 até P=32, e o speedup e a eficiência seguem o padrão clássico de strong scaling: eficiência de 100% em P=1 (por definição) caindo para a faixa de 6%–40% em P=32, dependendo do tamanho. Como antecipado na Seção 2.3, a eficiência tende a ser maior para matrizes maiores em um mesmo P por exemplo, em P=16: E=15,51% (N=512), 24,71% (N=1024), 41,49% (N=2048) e 60,00% (N=4096) — porque o trabalho local cresce mais rápido que o custo fixo das três coletivas, amortizando melhor a comunicação (efeito equivalente ao previsto pela Lei de Gustafson ao aumentar o tamanho do problema).

6.3 Duas anomalias observadas

Platô/recuo entre P=4 e P=8: para N=1024, 2048 e 8192, o tempo em P=8 é igual ou até ligeiramente pior que em P=4 (ex.: N=1024, 0,791 ms em P=4 contra 0,803 ms em P=8). Isso indica que, nessa faixa, o ganho de dividir o trabalho entre mais processos foi anulado pelo custo extra de comunicação/sincronização — um sinal de que o problema, para esse P, já estava próximo do ponto de saturação da comunicação para aquele tamanho.

Salto entre P=8 e P=16: em todos os cinco tamanhos de matriz, há uma queda abrupta e consistente de tempo ao passar de P=8 para P=16 (visível como um "degrau" na Figura 1), bem maior do que a tendência suave observada nos demais intervalos. Como esse salto aparece simultaneamente em todas as curvas independente do tamanho do problema, é mais provável que reflita uma característica de hardware/topologia do nó da partição amd-512 (por exemplo, uma fronteira de socket/domínio NUMA, ou de cache compartilhado, sendo cruzada ao alocar 16 processos) do que um efeito do algoritmo em si. Isso está fora do escopo do código MPI implementado, mas vale registrar como observação de engenharia de sistemas.

Consistente com essas duas observações, a eficiência não é estritamente decrescente com P: ela cai de P=1 a P=8, sobe de P=8 para P=16 (refletindo o salto acima) e volta a cair de P=16 para P=32 em todos os tamanhos — reforçando que o comportamento de desempenho aqui é moldado tanto pela Lei de Amdahl (overhead fixo de comunicação) quanto por efeitos de hardware específicos do nó utilizado.

6.4 Corretude

A corretude da implementação foi validada separadamente com a flag --verify erro absoluto máximo de 0,00e+00 entre o resultado paralelo e o cálculo serial de referência, testado com diferentes combinações de M, N e P, confirmando que a distribuição de linhas via Scatter, a difusão de x via Bcast e a reconstrução de y via Gather preservam exatamente o resultado matemático esperado — os tempos discutidos acima refletem unicamente custo de comunicação/computação, não erro de implementação.

7. Conclusão

O programa implementa exatamente a estratégia especificada no enunciado: distribuição da matriz A por linhas com MPI_Scatter, difusão do vetor x com MPI_Bcast, cálculo local independente por processo e recomposição do vetor y com MPI_Gather, e essa implementação foi validada como matematicamente correta: erro nulo frente ao cálculo serial. Desta vez a varredura foi concluída integralmente: as 30 combinações de tamanho de matriz 512 a 8192 e número de processos 1 a 32 planejadas no script.h foram todas executadas, com --ntasks=32.

A comparação por tamanho de matriz confirma a complexidade quadrática esperada ($O(N^2)$) para a multiplicação matriz-vetor densa quando $M=N$, com o tempo serial crescendo por um fator de $\approx 4\times$ a cada duplicação de N. A comparação por número de processos confirma o comportamento clássico de strong scaling, speedup sub-linear e eficiência decrescente com P, mais acentuado em matrizes pequenas e mais ameno em matrizes maiores, exatamente como previsto pela Lei de Amdahl e pela intuição de Gustafson discutidas na Seção 2.3. Duas anomalias pontuais: o platô entre $P=4$ e $P=8$, e o salto de desempenho entre $P=8$ e $P=16$, presente em todos os tamanhos foram identificadas e discutidas como prováveis efeitos de topologia de hardware do nó do NPAD, e não como falhas da implementação MPI, cuja corretude foi verificada de forma independente. No conjunto, os resultados corroboram tanto a implementação correta e fiel ao enunciado quanto a teoria de desempenho paralelo apresentada na fundamentação.

Anexo A — Código-fonte (multMxV.c)

```
/*
 * Tarefa 16 - Multiplicacao matriz-vetor distribuida com MPI
 * y = A * x, A: M x N (row-major), x: N, y: M
 *
 * Estrategia:
 * - A matriz A e distribuida por linhas entre os processos com MPI_Scatter
 *   (cada processo recebe local_M = M / nprocs linhas contiguas)
 * - O vetor x inteiro e distribuido a todos com MPI_Bcast
 * - Cada processo calcula os elementos de y correspondentes as suas linhas
 * - Os resultados parciais sao recolhidos no processo 0 com MPI_Gather
 *
 * Uso:
 * mpirun -np P ./mv_mpi M N [reps] [--verify]
 *
 * M, N      : dimensoes da matriz (M deve ser divisivel por P, pois
 *             MPI_Scatter exige blocos de tamanho igual)
 * reps      : numero de repeticoes usadas para medir o tempo medio
 *             (default = 10)
 * --verify  : depois das medicoes, o processo 0 recalcula y de forma
 *             serial e imprime o maior erro absoluto em relacao ao
 *             resultado paralelo, para validar a corretude
 */

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

static double *aloca_vetor(long n) {
    double *v = (double *) malloc((size_t) n * sizeof(double));
    if (!v) {
        fprintf(stderr, "Falha ao alocar %ld doubles\n", n);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    return v;
}

/* Gera valores pseudo-aleatorios deterministicos (mesma semente sempre),
 * para que as execucoes sejam reprodutíveis e comparáveis entre si. */
static void preenche_aleatorio(double *v, long n, unsigned int seed) {
    unsigned int state = seed;
    for (long i = 0; i < n; i++) {
        state = state * 1103515245u + 12345u;
        v[i] = (double) ((state >> 8) % 1000) / 100.0; /* [0, 10) */
    }
}

static void multiplica_serial(const double *A, const double *x, double *y,
                             long M, long N) {
    for (long i = 0; i < M; i++) {
        double soma = 0.0;
        const double *linha = A + i * N;
        for (long j = 0; j < N; j++) {
            soma += linha[j] * x[j];
        }
        y[i] = soma;
    }
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    if (argc < 3) {
        if (rank == 0) {
            fprintf(stderr,
                "Uso: %s M N [reps] [--verify]\n", argv[0]);
        }
    }
}
```

```

    }
    MPI_Finalize();
    return 1;
}

long M = atol(argv[1]);
long N = atol(argv[2]);
int reps = (argc >= 4 && argv[3][0] != '-') ? atoi(argv[3]) : 10;
int verify = 0;
for (int i = 3; i < argc; i++) {
    if (strcmp(argv[i], "--verify") == 0) verify = 1;
}

if (M % nprocs != 0) {
    if (rank == 0) {
        fprintf(stderr,
            "Erro: M=%ld precisa ser divisivel pelo numero de "
            "processos (nprocs=%d) para usar MPI_Scatter/MPI_Gather "
            "com blocos de tamanho igual.\n", M, nprocs);
    }
    MPI_Finalize();
    return 1;
}

long local_M = M / nprocs;

/* Apenas o processo 0 possui a matriz completa e o vetor y completo */
double *A = NULL, *y = NULL;
double *x = aloca_vetor(N); /* todos os processos terao x apos o Bcast */
double *local_A = aloca_vetor(local_M * N);
double *local_y = aloca_vetor(local_M);

if (rank == 0) {
    A = aloca_vetor(M * N);
    y = aloca_vetor(M);
    preenche_aleatorio(A, M * N, 42u);
    preenche_aleatorio(x, N, 7u);
}

double tempo_total = 0.0;

for (int r = 0; r < reps; r++) {
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    MPI_Scatter(A, (int)(local_M * N), MPI_DOUBLE,
        local_A, (int)(local_M * N), MPI_DOUBLE,
        0, MPI_COMM_WORLD);

    MPI_Bcast(x, (int) N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    for (long i = 0; i < local_M; i++) {
        double soma = 0.0;
        const double *linha = local_A + i * N;
        for (long j = 0; j < N; j++) {
            soma += linha[j] * x[j];
        }
        local_y[i] = soma;
    }

    MPI_Gather(local_y, (int) local_M, MPI_DOUBLE,
        y, (int) local_M, MPI_DOUBLE,
        0, MPI_COMM_WORLD);

    MPI_Barrier(MPI_COMM_WORLD);
    double t1 = MPI_Wtime();
    tempo_total += (t1 - t0);
}

if (rank == 0) {
    double tempo_medio = tempo_total / reps;
    /* Saida em CSV para facilitar coleta de dados nos benchmarks:
    * nprocs,M,N,reps,tempo_medio_s */
    printf("%d,%ld,%ld,%d,%.9f\n", nprocs, M, N, reps, tempo_medio);
}

```

```

    if (verify) {
        double *y_serial = aloca_vetor(M);
        multiplica_serial(A, x, y_serial, M, N);
        double erro_max = 0.0;
        for (long i = 0; i < M; i++) {
            double dif = fabs(y[i] - y_serial[i]);
            if (dif > erro_max) erro_max = dif;
        }
        fprintf(stderr,
            "[verify] erro absoluto maximo entre paralelo e serial: %e\n",
            erro_max);
        free(y_serial);
    }
}

free(x);
free(local_A);
free(local_y);
if (rank == 0) {
    free(A);
    free(y);
}

MPI_Finalize();
return 0;
}

```

Anexo B — Script SLURM (script.sh)

```
#!/bin/bash
#SBATCH --job-name=tarefa16
#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH --ntasks=32
#SBATCH --time=00:30:00
#SBATCH --output=tarefa16_%j.out
#SBATCH --error=tarefa16_%j.err

set -e

mpicc -O3 -o multMxV multMxV.c -lm

REPS=30
OUT="resultados_tarefa16_${SLURM_JOB_ID}.csv"
echo "nprocs,M,N, reps,tempo_medio_s" > "$OUT"

TAMANHOS=(512 1024 2048 4096 8192)
NPROCS_LIST=(1 2 4 8 16 32)

for M in "${TAMANHOS[@]"; do
    N=$M
    for P in "${NPROCS_LIST[@]"; do
        if [ $((M % P)) -ne 0 ]; then
            continue
        fi
        mpirun -np "$P" ./multMxV "$M" "$N" "$REPS" >> "$OUT"
    done
done

echo "Concluido. Resultados em $OUT"
```